Authentication Decision Guide

Use this guide to choose your user identification strategy, identity provider, and token flow architecture. Each decision compares the available options, their tradeoffs, and the situations they fit. Work through the choices for each user population, then use the linked technical docs to implement your approach.

Companion to the Authentication and User Management docs. Builds on Identity Management: A Practical Guide.

Authentication establishes **who a user is**. Authorization determines **what that user can access**. This guide focuses on authentication; use the Access Control Decision Guide for tenancy and permissions.

Your staff, partner clinicians, patients, and caregivers may need different approaches. Identify those populations and any existing identity systems before making the three decisions below.

Decision 1: What is your user identification strategy?

Choose the identifier that connects a login to the correct Medplum account. This is separate from choosing an identity provider: an external IDP can still use email matching.

Questions: Is email unique for each user? How often does it change? Can different people share a contact email? Do you already have a stable identifier from an IDP?

Option A: Email matching

Medplum uses email matching by default, including when authenticating through an external IDP.

- **Advantages:**
  - Simple to configure when each person has a distinct email and changes are infrequent.
  - Works with external IDPs as well as Medplum-managed authentication.
- **Tradeoffs:**
  - Email changes must also be reflected in the Medplum login account. Updating only the Patient or Practitioner contact email is insufficient.
  - Shared addresses cannot reliably distinguish separate people.
- **Best fit:** Applications with straightforward identity requirements and an owner for occasional account updates.

Option B: External ID matching

Use the IDP's subject identifier to connect the external identity to Medplum. Configure subject-based matching (`useSubject`) and provision the corresponding external ID.

- **Advantages:**
  - Identity can remain stable when a person's email changes.
  - Supports distinct identities independently of contact email.
- **Tradeoffs:**
  - Provisioning and login must use the same mapping; changing IDPs requires a migration plan.
  - Subject values belong to a provider context. Do not assume they are globally unique across unrelated IDPs.
- **Best fit:** An existing IDP supplies stable identifiers, email changes are common, or email does not uniquely identify users.

Verify the exact subject format against the selected Medplum flow before bulk provisioning.

Recommended approach

Situation	Approach
Distinct emails and manageable account changes	Start with email matching. An external IDP alone is not a reason to change the matching key.
Established IDP identity that must survive email changes	Use external ID matching and define who maintains the mapping.
Staff and patients have different identity requirements	Choose separately for each population. For example, staff may use corporate subject IDs while patients use email matching.
Separate people share a contact email	Use distinct authenticated identities and evaluate external ID matching. Also design recovery and MFA so mailbox access does not allow recovery of both accounts.
Caregiver accesses another person's records	Give the caregiver their own identity and explicit access. A shared email is not evidence of authorization.

Record: Matching key for each population, mapping owner, and procedures for email changes and IDP migration.

References: Using External IDs, Updating User Emails.

Decision 2: What is your identity provider strategy?

Choose who manages credentials and how users reach the correct identity provider (IDP).

Questions: Do you already use an IDP? Does everyone in an application use the same provider? Do customer organizations bring their own SSO? Does an existing identity broker already handle that routing?

Option A: Medplum-managed authentication

Medplum manages user credentials, password resets, and authentication. Your application uses its supported login flows and components.

- **Advantages:**
  - No separate IDP integration is needed.
  - Medplum provides credential management and supported login components.
- **Tradeoffs:**
  - Your product still needs decisions about registration, invitations, MFA, and recovery.
  - This does not satisfy a requirement to use an existing corporate IDP by itself.
- **Best fit:** Applications without an existing identity system or external SSO requirement.

Implementation: Redirect login or Embedded Login. Choose the login presentation separately from who manages credentials.

Option B: External IDP with application-based routing

Configure an external identity provider on the Medplum `ClientApplication`. The application determines which provider is used, such as one IDP for the patient portal and another for the clinician application.

- **Advantages:**
  - Reuses existing identity infrastructure.
  - Different applications can use different providers.
- **Tradeoffs:**
  - If users of the same application need different upstream IDPs, define how that routing will happen, such as through an identity broker.
- **Best fit:** The application selects a known IDP or identity broker for its users.

Implementation: External Identity Providers.

Option C: External IDP with domain-based routing

Use Medplum `DomainConfiguration` to route users to an IDP based on their email domain.

- **Advantages:**
  - An organization's SSO applies across Medplum applications, including the Medplum App.
- **Tradeoffs:**
  - Applies to all users of the domain, including developer accounts.
  - Hosted setup requires Enterprise and Medplum assistance; self-hosted setup requires super admin privileges.
- **Best fit:** Corporate email domains should determine the IDP consistently across Medplum applications.

Implementation: Domain-level Identity Providers.

Recommended approach

Use Medplum-managed authentication when it meets the product's requirements. If an external IDP is required, use application-based routing when the application selects the provider and domain-based routing when the user's email domain should select it across Medplum applications.

If an existing identity broker already routes among customer IDPs, retain that routing and evaluate token exchange in Decision 3. Customer-controlled routing is separate from Medplum's domain-level configuration.

Domain routing selects where a user authenticates. It does not establish tenant isolation or grant access to patient records.

Record: Credential owner and routing mechanism for each population.

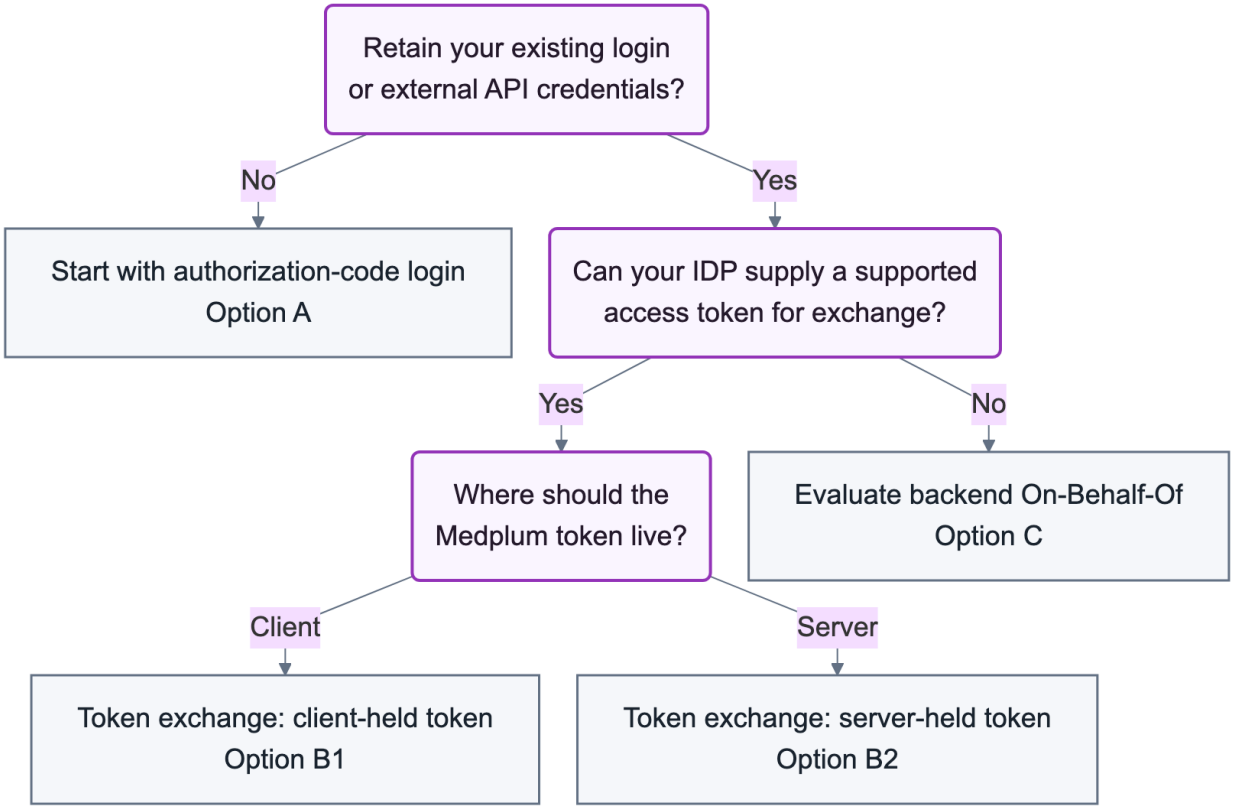
Decision 3: What is your token flow architecture?

Choose how your application obtains a token for Medplum and where it uses that token. An external IDP access token and a Medplum access token serve different APIs; do not assume they are interchangeable.

Questions: Does your application already authenticate users? Does the client need an external token for other services? Must it call Medplum directly, or should all Medplum access pass through your backend?

"Client" below means your browser or native application. "Server" means your trusted application backend. The options below primarily address external or existing authentication. If you chose Medplum-managed authentication, choose redirect login or embedded login, then resolve the additional decisions below.

Use this tree as a starting point for external-IDP user authentication. The options below explain the tradeoffs; specialized flows such as self-hosted direct external authentication are covered under Additional Decisions.



Option A: External IDP authorization-code flow

Often called **three-legged OAuth**, this flow lets Medplum handle the external authentication callback and issue Medplum tokens to your application.

1. The user starts login and authenticates with the external IDP.

2. The IDP redirects to Medplum's external authentication callback.
3. Medplum redirects to your application with a Medplum authorization code.
4. Your application exchanges that code for Medplum tokens.

- **Advantages:**
  - The client can call Medplum directly without retaining an external access token for this flow.
  - Fits direct SDK and authenticated component use without a proxy for Medplum requests.
- **Tradeoffs:**
  - Requires a redirect/callback flow and coordination of redirect settings with the IDP.
  - Does not give the client an external access token to reuse with other services. An application with an established external login may fit token exchange better.
  - A Medplum logout does not by itself define what happens to the external IDP's SSO session; choose whether users should be signed out there too.
- **Best fit:** A redirect login fits the application and Medplum is the primary client API.

Implementation: External Identity Providers.

Option B: Token exchange

Your application already has an external access token. Medplum validates it through the configured IDP and issues a Medplum token for the resolved user and membership. The configured provider must support validating that access token through its UserInfo endpoint.

- **Advantages:**
  - Reuses a valid external login without a second interactive login to Medplum.
  - Lets your application retain its existing login and IDP-routing logic.
  - Produces a user-scoped Medplum token that can be used in either the client or backend.

- **Tradeoffs:**
  - Requires a supported external access token and configured UserInfo validation. An ID token or arbitrary application session cookie is not an interchangeable input.
  - You must coordinate external and Medplum credential lifecycles and select the intended membership.
  - Other APIs must accept the appropriate credentials for their own audience and scopes; token exchange does not make one external token valid for every service.
- **Best fit:** An existing application owns login, or its client also needs external authentication for other services.

The presence of other integrations alone does not require token exchange. If those integrations run in Bots or your backend using service credentials, the client may still need only Medplum authentication.

Where will you store the Medplum token?

Once you choose token exchange, decide between these two placements. Both produce **user-scoped Medplum tokens**. The choice changes where credentials are managed and which system carries requests to Medplum.

B1: Keep the Medplum token in the client

The client obtains a Medplum token through exchange and calls Medplum directly.

- **Advantages:**
  - Direct SDK/API access and authenticated React components fit naturally.
  - WebSocket subscriptions connect directly to Medplum.
  - No application proxy is needed for those requests.
- **Tradeoffs:**
  - The client manages both external and Medplum credentials, including expiry, refresh, and logout.
  - Medplum credentials are present in the client. Choose storage and session controls appropriate to the browser or native platform.

    - Every direct API request must be constrained by Medplum access policies; UI restrictions cannot enforce permissions.
- **Best fit:** Direct Medplum access is valuable and the client can manage both credential lifecycles.

B2: Keep the Medplum token on the server

Your backend exchanges the authenticated user's external token, retains the resulting Medplum token, and uses it to fulfill the user's requests. The client keeps its existing application authentication mechanism.

- **Advantages:**
    - Medplum credentials stay out of the client, so the client does not need to manage a second API credential lifecycle.
    - Fits an existing backend that coordinates multiple services or deliberately mediates all data access.
    - Medplum still enforces the resolved user's access policy on the backend's requests.
- **Tradeoffs:**
    - Credential renewal and logout work move to the backend; they do not disappear.
    - Your team operates the proxy and per-user token cache. This adds a request hop and another service whose availability and capacity affect Medplum access.
    - Live updates need a path from Medplum through your backend to the client. Components expecting a directly authenticated Medplum client need adaptation.
    - Every request must resolve to the correct user, membership, and cached token. A shared service token is not a substitute.
- **Best fit:** Backend mediation or keeping Medplum credentials on the server is an explicit architectural requirement, and your team can operate that layer.

For server storage, bind cached tokens to the authenticated identity, membership, and session context. Validate incoming requests and reject inconsistent mappings. Never reuse another user's token or silently fall back to an application credential when user authentication fails.

Implementation: Token Exchange.

Option C: On-Behalf-Of

Your backend authenticates to Medplum as an admin `ClientApplication`, then supplies `X-Medplum-On-Behalf-Of` with the user's membership context on each request. Medplum uses that membership's access policy for the request.

- **Advantages:**
  - Supports a backend that owns user authentication but cannot exchange a supported external access token.
  - Retains user attribution and policy enforcement when correctly configured.
- **Tradeoffs:**
  - Requires an application with Project Admin rights and a trusted identity-to-membership mapping.
  - Without the header, requests use the application's own access. A wrong valid membership can act for the wrong user.
  - Your backend is responsible for authenticating the person and choosing the delegation context; the header itself is not proof of the user's identity.
- **Best fit:** A legacy backend requires this delegation model and cannot use user-scoped token exchange.

This differs from server-side token exchange: the underlying credential represents the **application**, not the user. Reject missing or inconsistent user context, prevent callers from supplying arbitrary delegation headers, and separate privileged service operations from user requests. Do not assume an access policy eliminates the admin capability this feature requires.

Obtain the backend's application token with Client Credentials. Include the delegation header on each user request; it does not persist across requests. Follow the On-Behalf-Of documentation for attribution metadata and configuration.

Recommended approach

Situation	Approach
External IDP, redirect login, and primarily Medplum client access	Start with the authorization-code flow.
Existing external login and direct Medplum client access	Use token exchange with Medplum tokens in the client.
Existing external login and backend-mediated Medplum access	Use token exchange with Medplum tokens on the server.
Backend can use either user-scoped exchange or On-Behalf-Of	Prefer user-scoped exchange to avoid depending on a privileged application credential and a delegation header.
Legacy authentication prevents supported token exchange	Evaluate On-Behalf-Of with explicit ownership of authentication, mapping, and delegation controls.

Record: Token flow, token location, membership-selection rules, and ownership of proxy operations and credential lifecycles.

Additional Decisions

Resolve these after choosing the main architecture. Use the technical docs for configuration details.

How will you provision users and select memberships?

A `Patient` or `Practitioner` alone does not create a login. Provision the `User`, `ProjectMembership`, and profile relationship, and resolve links to existing clinical records before granting access.

Situation	Approach
Patients and clinicians primarily use one production project	Explicitly choose project-scoped users; the Practitioner default is server-scoped.
Developers or administrators need multiple projects	Evaluate server-scoped users and the required memberships. Hosted administrative control differs by user scope.
Token exchange has multiple candidate memberships	Pass the intended `membership_id` and verify its relationship to the authenticated identity. Do not depend on first-membership selection.
Exchange must reach another project	A per-`ClientApplication` exchange is limited to that client's project. Self-hosted server-level configuration supports broader selection; discuss hosted requirements with Medplum.

References: User Management, User Scopes, Token Exchange.

What are your MFA and session lifecycle requirements?

Choose which system enforces MFA, who can recover an account, and whether logout affects one device or all sessions. Medplum's documented MFA challenge applies to username/password login; enforce external-login MFA in the IDP.

If the client holds both external and Medplum credentials, address both lifecycles and any refresh credentials. If Medplum tokens stay on the server, terminate the application session, invalidate cached credentials, and revoke associated Medplum sessions according to your policy. Clearing client storage alone is not revocation, and disabling an IDP account does not necessarily invalidate already-issued Medplum tokens immediately.

References: MFA, Access Token Management, Logout.

Do you need a specialized flow?

Situation	Approach
Background service acts as itself	Use Client Credentials with a narrowly scoped `ClientApplication`. Prefer short-lived tokens to sending long-lived credentials on every FHIR request with Basic auth.
Browser or native authorization-code client	Use PKCE and documented redirect protections. Native apps should authorize through the system browser. Never embed a confidential client secret in browser code or a mobile binary. See OAuth security guidance and native-app guidance.
Self-hosted deployment must accept external JWTs directly	Evaluate Direct External Authentication. Confirm server configuration, claim mapping, uniqueness, and revocation/cache behavior. No Medplum token exchange is performed.
Third-party app needs a SMART launch	Use the SMART on FHIR guidance; distinguish it from your first-party login.
Activation precedes account creation	Follow the mobile activation guidance, keeping activation narrowly scoped and separate from normal sessions.

Before You Implement

Record the three architecture choices for each user population, then test first login, email change, account recovery, membership switching, token expiry, logout, and offboarding. For proxies, verify that missing user context fails without falling back to application credentials.